UNIVERSITÀ DI PARMA

Camera Models (2)

Summary

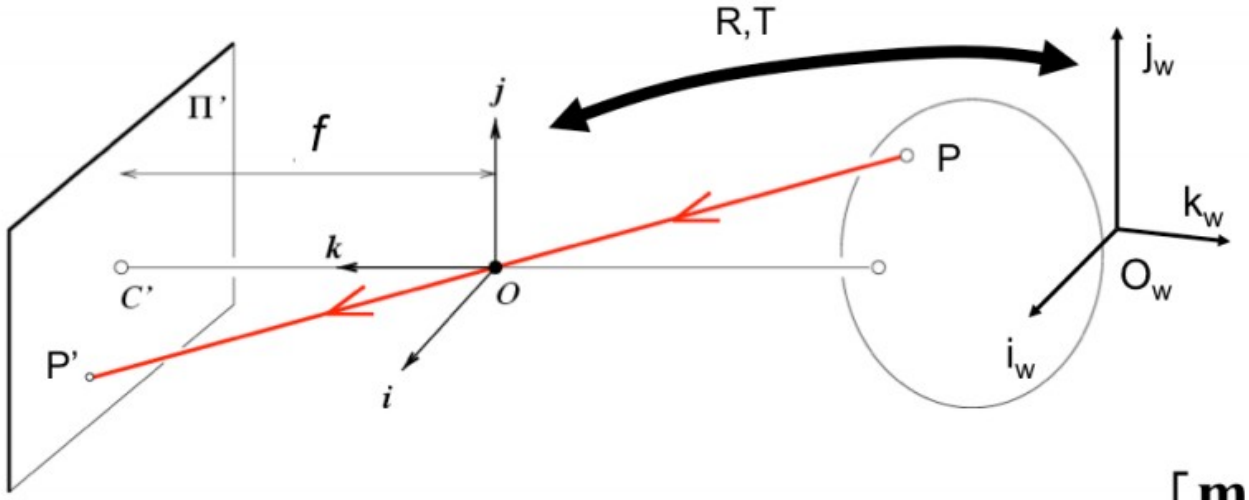
UNIVERSITÀ DI PARMA

- Pin Hole Camera recap
- Calibration

UNIVERSITÀ DI PARMA

Pin Hole Model recap

Perspective Transformation



$$P'_{3\times1} = M\,P_w = K_{3\times3}\begin{bmatrix} R & T \end{bmatrix}_{3\times4} P_{w\,4\times1} \qquad M = \begin{bmatrix} \mathbf{m}_1 \\ \mathbf{m}_2 \\ \mathbf{m}_3 \end{bmatrix}$$

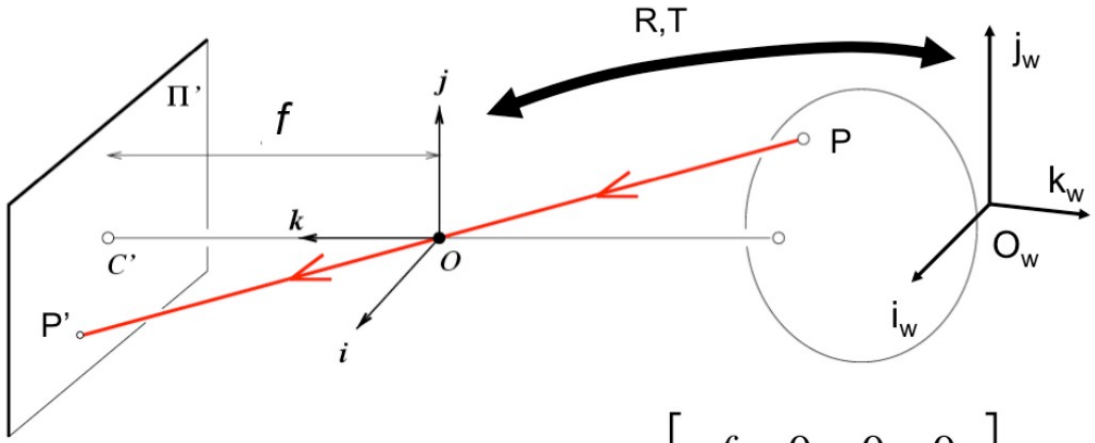
$$= \begin{bmatrix} \mathbf{m}_1 \\ \mathbf{m}_2 \\ \mathbf{m}_3 \end{bmatrix} P_W = \begin{bmatrix} \mathbf{m}_1 P_W \\ \mathbf{m}_2 P_W \\ \mathbf{m}_3 P_W \end{bmatrix}$$

$$\mathbf{E} \rightarrow \left(\frac{\mathbf{m}_1 P_w}{\mathbf{m}_3 P_w}, \frac{\mathbf{m}_2 P_w}{\mathbf{m}_3 P_w}\right) \quad \text{[Eq.12]}$$

Courtesy: Silvio Savarese

Perspective Transformation (ideal case)

- Simplified situation: no rotation, no translation, no skew, no offset, squared pixels



$$M = K\begin{bmatrix} R & T \end{bmatrix} = K\begin{bmatrix} I & 0 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$\rightarrow P'_E = \left(\frac{\mathbf{m}_1 P_w}{\mathbf{m}_3 P_w}, \frac{\mathbf{m}_2 P_w}{\mathbf{m}_3 P_w}\right) = \left(f\frac{x_w}{z_w}, f\frac{y_w}{z_w}\right) \qquad P_w = \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix}$$

Courtesy: Silvio Savarese

Perspective Transformation

- 11 degrees of freedom

$$P' = M P_w = K[R \quad T] P_w$$

Internal parameters

External parameters

$$\mathcal{M} = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix}_{3\times 4}$$

$$K = \begin{bmatrix} \alpha & -\alpha\cot\theta & u_o \\ 0 & \frac{\beta}{\sin\theta} & v_o \\ 0 & 0 & 1 \end{bmatrix} \qquad R = \begin{bmatrix} \mathbf{r}_1^T \\ \mathbf{r}_2^T \\ \mathbf{r}_3^T \end{bmatrix} \qquad T = \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

Courtesy: Silvio Savarese

UNIVERSITÀ DI PARMA

Calibration
Tsai approach (1987)

Calibration

$$P' = M P_w = K[R \quad T] P_w$$

Internal parameters

External parameters

- Calibration ⟶ **estimation of intrinsic/extrinsic parameters**
- We need 1 or, rather, more images

Change notation:
$P = P_w$

$p = P'$

Calibration

UNIVERSITÀ DI PARMA

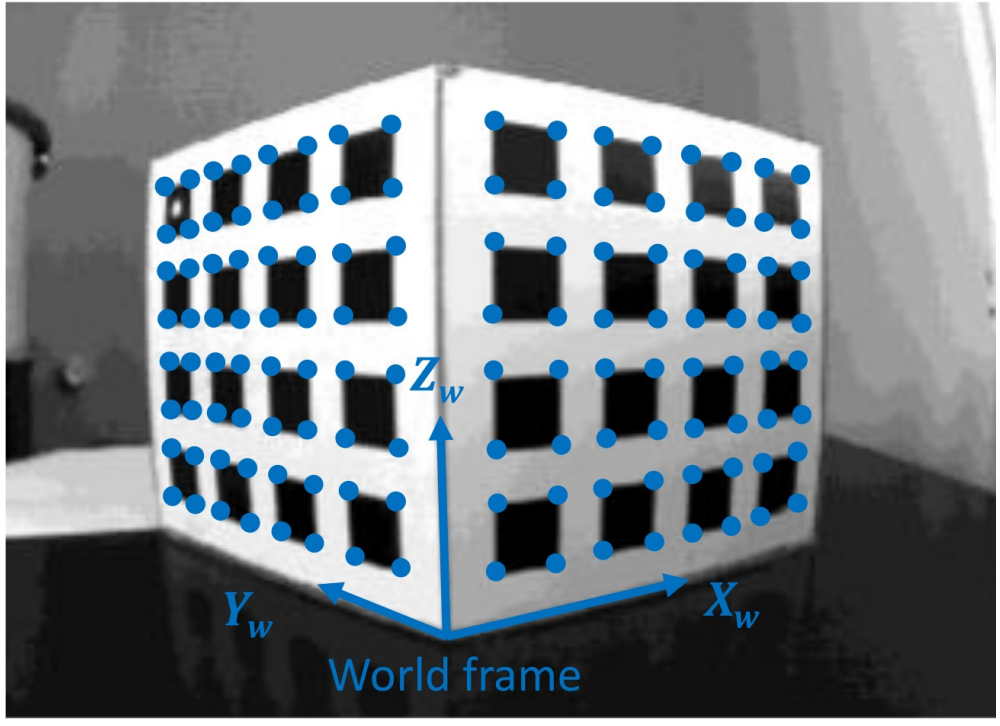
- Small notation change
  - $P_W \rightarrow P$
  - $P' \rightarrow p$

$$p = K\left[R\,T\right]P = MP$$

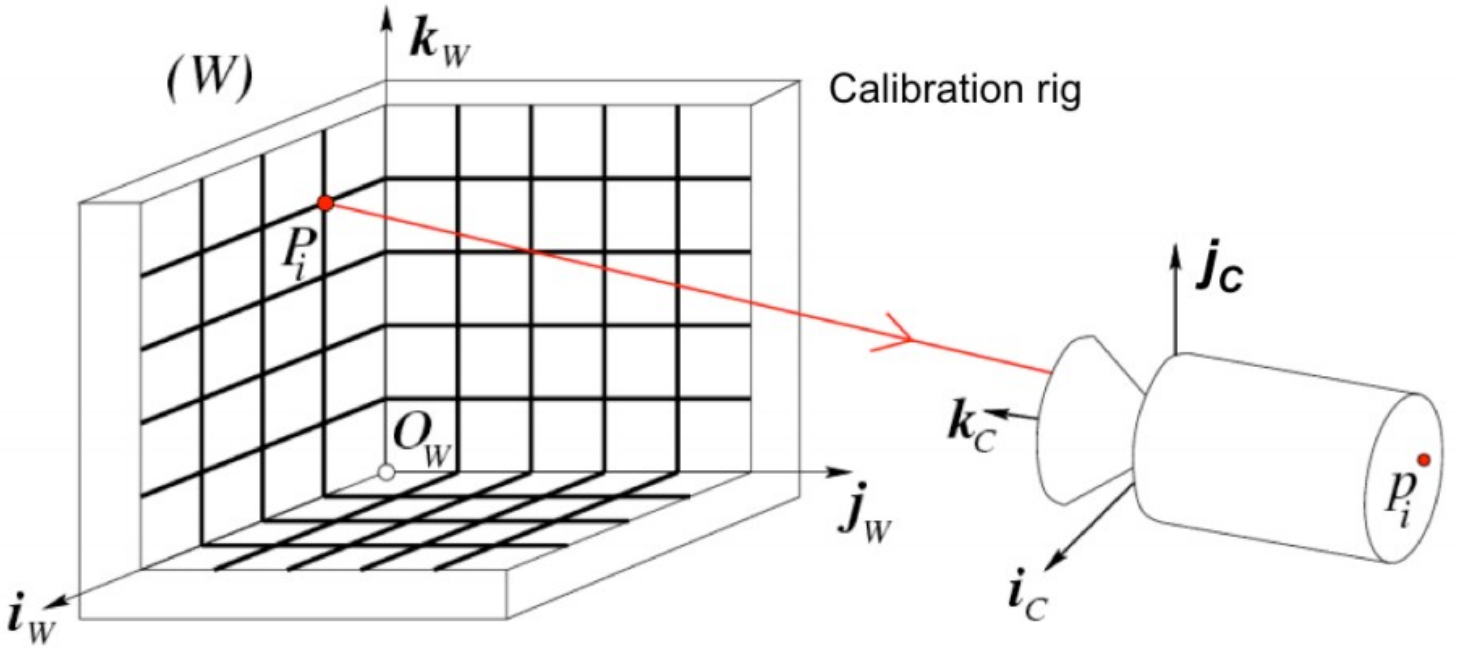
Tsai's method

UNIVERSITÀ DI PARMA

- We use 3D calibration targets and their projection on an image



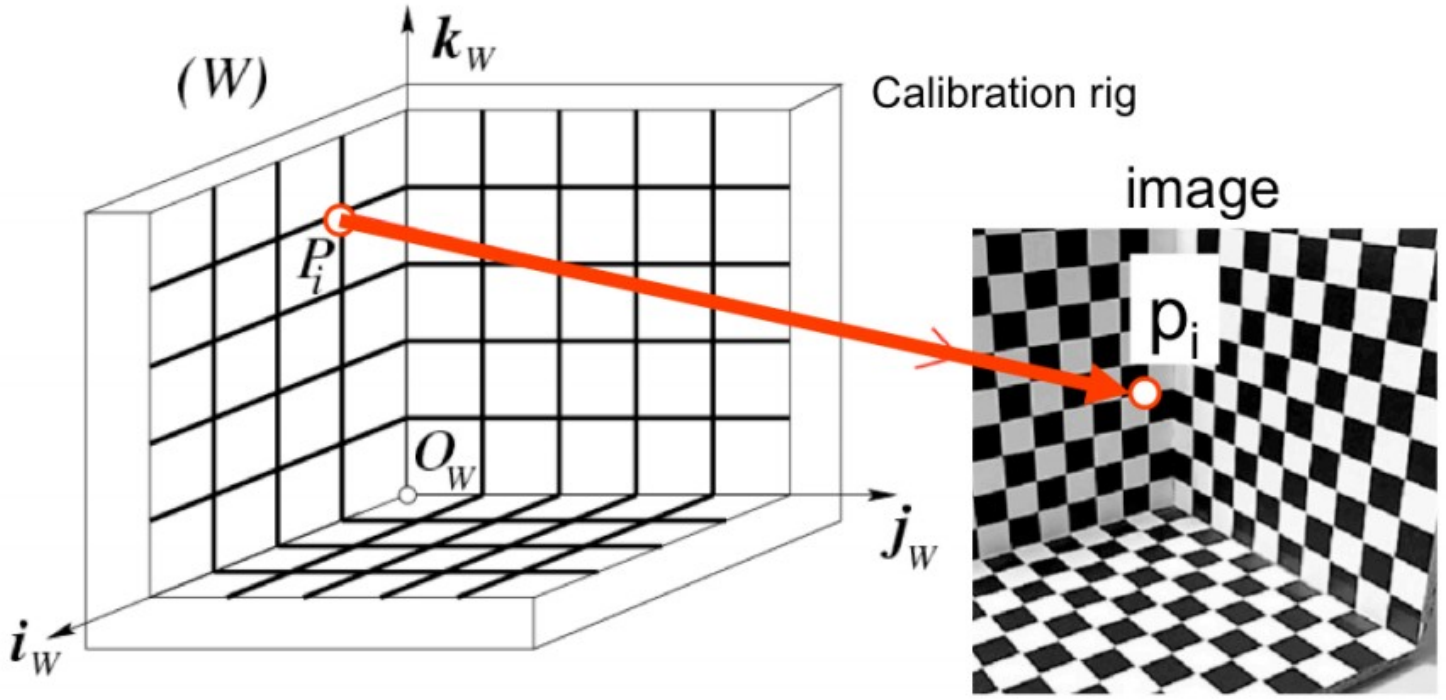
Calibration



- We need: n points (P_1, P_2,…,P_n) in known positions [O_w,i_w,j_w,k_w] in the world reference system

Courtesy: Silvio Savarese

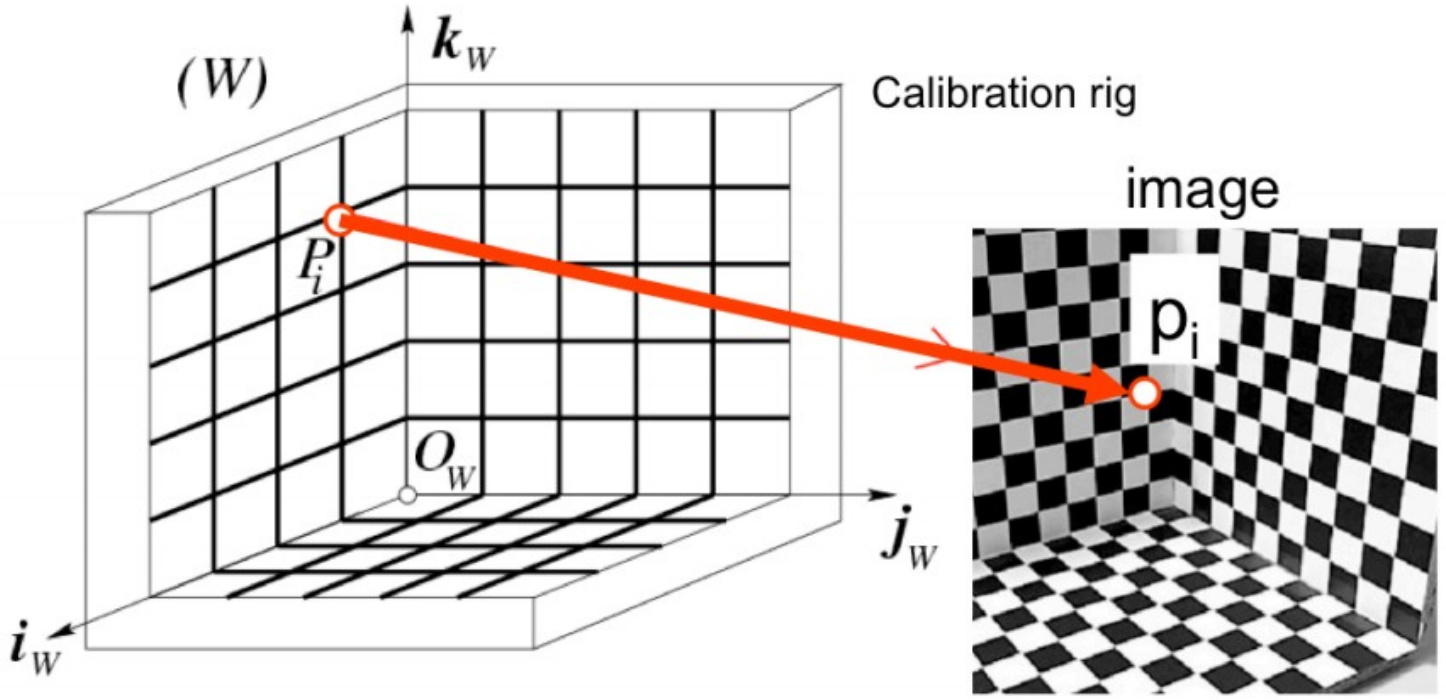
Calibration



- We also need: camera coordinates for the n points projections (p_1, p_2,…,p_n) as (i_c,j_c)

Courtesy: Silvio Savarese

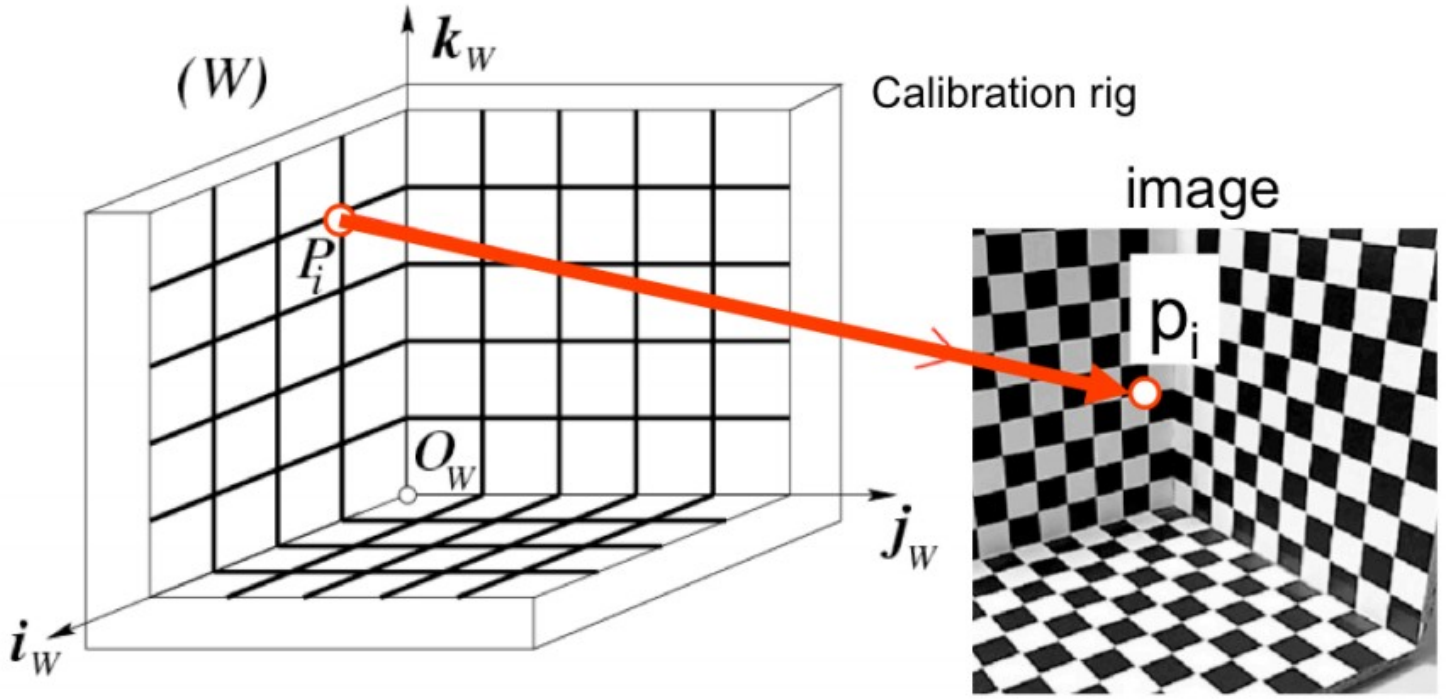
Calibration



- 11 degrees of freedom ⟶ how many matches we need?
- Each correspondence ⟶ 2 equations (world coordinates ⟶ row/column)
- We need, at least, 6 correspondences

Courtesy: Silvio Savarese

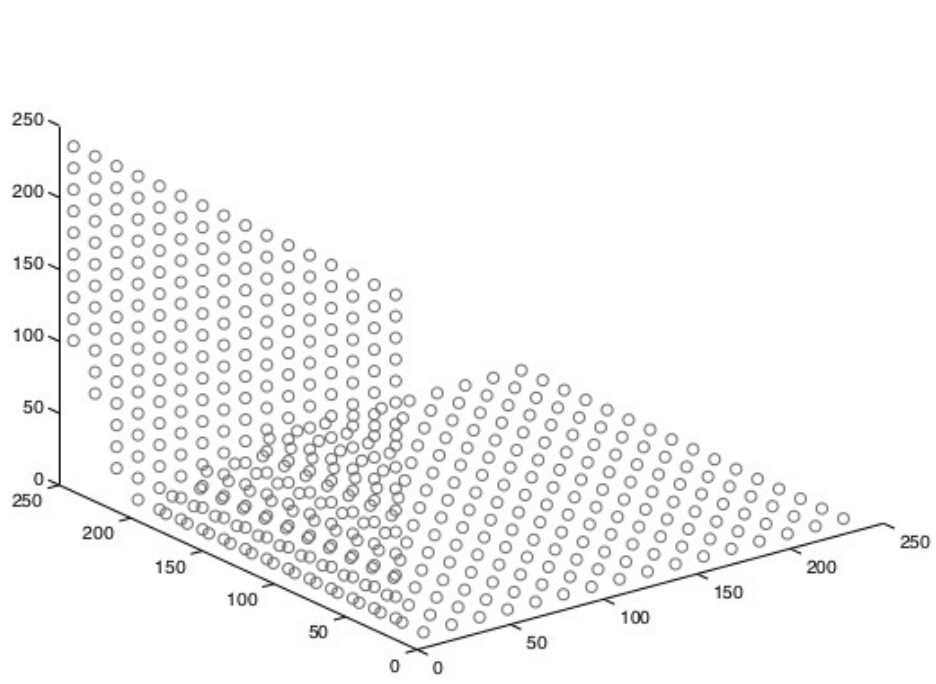
Calibration

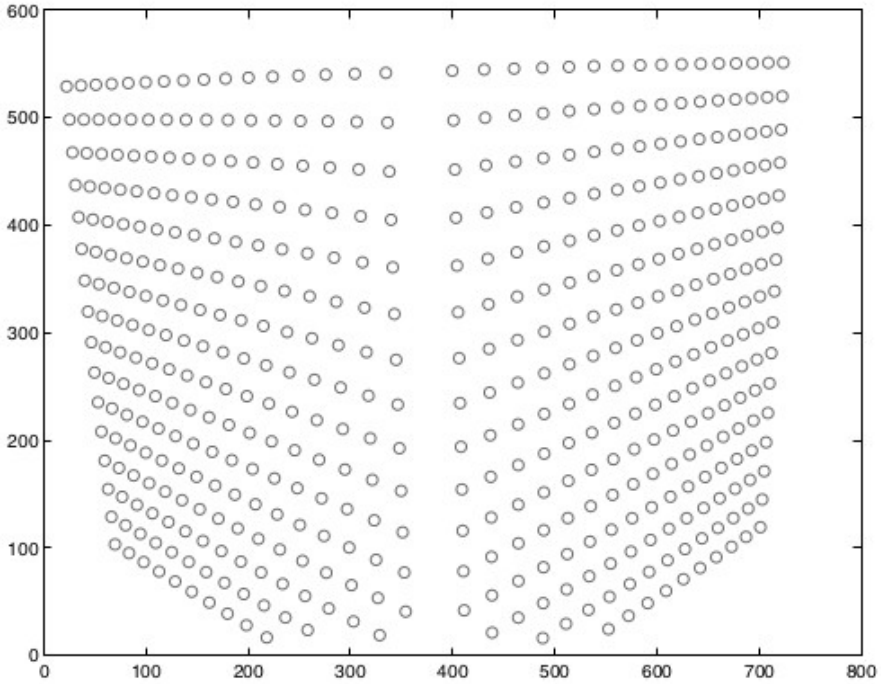


- Practically we need more than 6 correspondences!
- Consider error in measurements + digitalization error!

Courtesy: Silvio Savarese

Example of calibration data





- Example of 491 3D points
  - From: Janne Heikkilä; data copyright 2000, University of Oulu.

DLT Algorithm

UNIVERSITÀ DI PARMA

- 12 elements but actually there is a dependence among them → 11 unknowns

$$p = K \left[R \, T \right] P = MP$$

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

DLT Algorithm

UNIVERSITÀ DI PARMA

- Direct Linear Transform: rewrite the perspective projection equation as homogeneous system of linear equations

$$p = K \left[R \, T \right] P = MP$$

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

Calibration

UNIVERSITÀ DI PARMA

$$u_i = \frac{m_{00} X_i + m_{01} Y_i + m_{02} Z_i + m_{03}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}} \qquad v_i = \frac{m_{10} X_i + m_{11} Y_i + m_{12} Z_i + m_{13}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}}$$

$$m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23} - m_{00} u_i X_i - m_{01} u_i Y_i - m_{02} u_i Z_i - m_{03} u_i = 0$$

$$m_{10} X_i + m_{11} Y_i + m_{12} Z_i + m_{13} - m_{20} v_i X_i - m_{21} v_i Y_i - m_{22} v_i Z_i - m_{23} v_i = 0$$

We can recover 2 equations for each correspondence world ↔ image

Calibration

UNIVERSITÀ DI PARMA

Homogeneous System

$$\begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1X_1 & -u_1Y_1 & -u_1Z_1 & -u_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1X_1 & -v_1Y_1 & -v_1Z_1 & -v_1 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_nX_n & -u_nY_n & -u_nZ_n & -u_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_nX_n & -v_nY_n & -v_nZ_n & -v_n \end{bmatrix} \begin{matrix} m_{00} \\ m_{01} \\ m_{02} \\ m_{03} \\ m_{10} \\ m_{11} \\ m_{12} \\ m_{13} \\ m_{20} \\ m_{21} \\ m_{22} \\ m_{23} \end{matrix} = \begin{matrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{matrix}$$

Calibration

UNIVERSITÀ DI PARMA

- Step back and write in a simpler form

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_0 \\ m_1 \\ m_2 \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix} = \begin{bmatrix} m_0 \\ m_1 \\ m_2 \end{bmatrix} P_i$$

Calibration

UNIVERSITÀ DI PARMA

- Step back and write in a simpler form

$$u_i = \frac{m_{00} X_i + m_{01} Y_i + m_{02} Z_i + m_{03}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}} \qquad v_i = \frac{m_{10} X_i + m_{11} Y_i + m_{12} Z_i + m_{13}}{m_{20} X_i + m_{21} Y_i + m_{22} Z_i + m_{23}}$$

$$u_i = \frac{m_0 P_i}{m_2 P_i} \qquad v_i = \frac{m_1 P_i}{m_2 P_i}$$

$$-m_0 P_i + u_i (m_2 P_i) = 0 \qquad -m_1 P_i + v_i (m_2 P_i) = 0$$

Calibration

UNIVERSITÀ DI PARMA

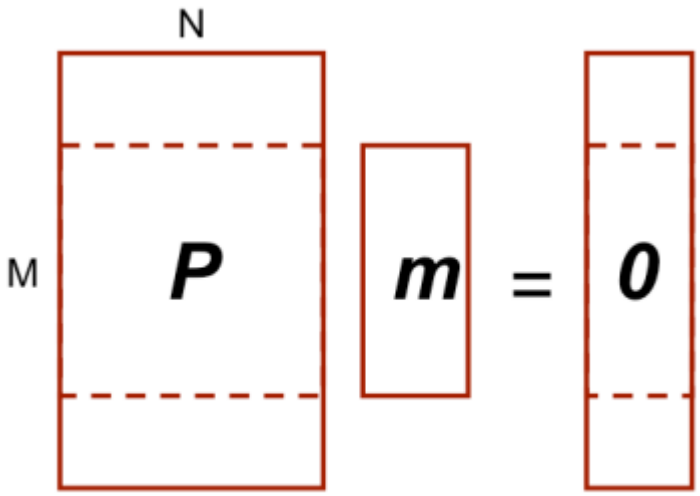
$$-m_0 P_i + u_i (m_2 P_i) = 0 \qquad -m_1 P_i + v_i (m_2 P_i) = 0$$

$$\begin{bmatrix} -P_0^T & 0^T & u_0 P_0^T \\ 0^T & -P_0^T & v_0 P_0^T \\ \dots & \dots & \dots \\ -P_n^T & 0^T & u_n P_n^T \\ 0^T & -P_n^T & v_n P_n^T \end{bmatrix}_{2n \times 12} \begin{bmatrix} m_0^T \\ m_1^T \\ m_2^T \end{bmatrix}_{12 \times 1} = \begin{bmatrix} 0 \\ \dots \\ 0 \end{bmatrix} \Rightarrow Pm = 0$$

Homogeneous System → 2n equations

Unknowns → m_i

Calibration



- M>N as we suggested
  - Rectangular system of equations
  - Overdetermined
    - "potentially" no solution other trivial one m=0
- The idea is to find the best fitting solution
- Minimize $|\mathrm{P}\,m|^2$
  - Assuming m != 0 $\longrightarrow$ $|m|^2 = 1$

Courtesy: Silvio Savarese

Recap: inverting a non-square matrix

UNIVERSITÀ DI PARMA

- A linear equation

$$Ax = y$$

- Can be solved finding a matrix B (ideally $B=A^{-1}$, namely $AB=I$)

$$x = By$$

- When A is taller than it is wide → possibly no solutions
- When A is wider than it is tall → multiple solutions

Singular Value Decomposition (SVD)

UNIVERSITÀ DI PARMA

- Partially generalize matrix inversion for non square matrices
  - Do you remember something?
- Each matrix can be factorized into **singular vectors** and **singular values**
- More precisely each matrix A can be seen as product of three matrices:

$$A = UDV^T$$

Singular Value Decomposition (SVD)

UNIVERSITÀ DI PARMA

- Supposing A as $m \times n$ matrix, U is a $m \times m$ matrix, D is a $m \times n$ one and V is a $n \times n$ one
- U and V orthogonal matrices
  - Namely $UU^T=U^TU=I$ and $VV^T=V^TV=I$
  - Columns are the left (U) or right (V) singular vectors
- D is a diagonal matrix
  - Elements along the diagonal are the singular values

Singular Value Decomposition (SVD)

UNIVERSITÀ DI PARMA

- A linear equation

$$Ax = y$$

- Can be solved finding a matrix B (ideally $B=A^{-1}$)

$$x = By$$

- When A is taller than it is wide → possibly no solutions
- When A is wider than it is tall → multiple solutions

Moore-Penrose pseudoinverse

UNIVERSITÀ DI PARMA

- The pseudoinverse of a matrix A or A^+ is defined as

$$A^+ = \lim_{\alpha \to 0} \left(A^T A + \alpha I \right)^{-1} A^T$$

- Practically A^+ can be approximated using SVD

$$A^+ = V D^+ U^T$$

- When A is taller than it is wide this allows to compute the best approximation

Calibration

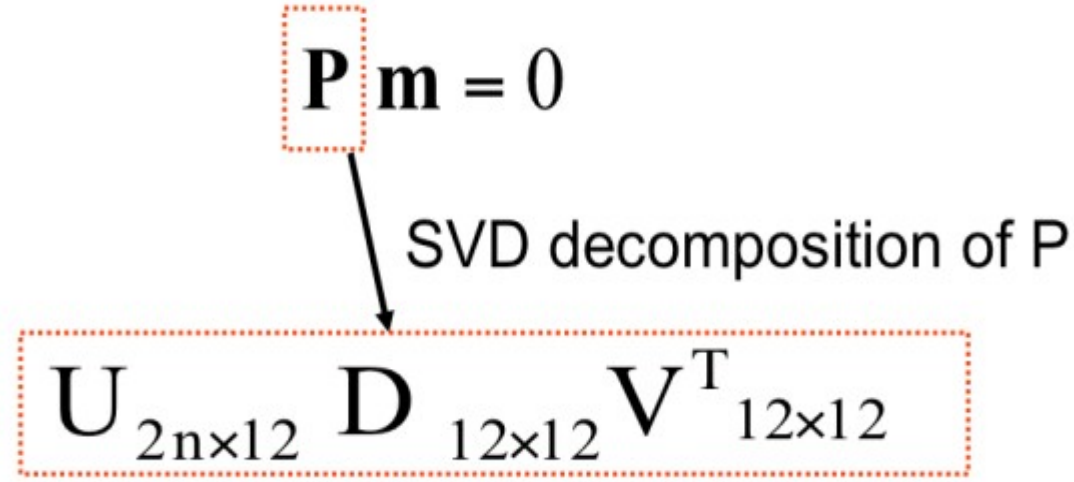
UNIVERSITÀ DI PARMA

- How we can solve such system?

$$\mathrm{P}\,m=0$$

- Via SVD decomposition!
  - This leads to the optimal solution assuming $|m|^2=1$

Calibration



- The last V column gives us m

$$\boldsymbol{m} \overset{\text{def}}{=} \begin{pmatrix} \mathbf{m}_1^T \\ \mathbf{m}_2^T \\ \mathbf{m}_3^T \end{pmatrix} \quad \Rightarrow \quad M$$

Courtesy: Silvio Savarese

Parameters Extraction

UNIVERSITÀ DI PARMA

Courtesy: Silvio Savarese

$$M = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix} \rho$$

- We can recover parameters but with a scale factor ρ
- Actually not an unknown, we can impose $|m|=1$
  - or equivalent Froebius norm $||M||=1$
- Extrinsic and Intrinsic parameters are separately extracted

Parameters Extraction

$$\frac{M}{\rho} = \begin{pmatrix} \alpha \mathbf{r}_1^T - \alpha \cot\theta \mathbf{r}_2^T + u_0 \mathbf{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta}\mathbf{r}_2^T + v_0 \mathbf{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \mathbf{r}_3^T & t_z \end{pmatrix} = \frac{K}{\rho}\begin{bmatrix} R & T \end{bmatrix}$$

$$K = \begin{bmatrix} \alpha & -\alpha\cot\theta & u_o \\ 0 & \frac{\beta}{\sin\theta} & v_o \\ 0 & 0 & 1 \end{bmatrix}$$

$A_{3\times3}$ $\quad$ $\mathbf{b}_{1\times3}$

Box 1

$$A = \begin{bmatrix} \mathbf{a}_1^T \\ \mathbf{a}_2^T \\ \mathbf{a}_3^T \end{bmatrix} \qquad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Estimated values

Intrinsic

$$\rho = \frac{\pm 1}{|\mathbf{a}_3|} \qquad u_o = \rho^2(\mathbf{a}_1 \cdot \mathbf{a}_3)$$

$$v_o = \rho^2(\mathbf{a}_2 \cdot \mathbf{a}_3)$$

$$\cos\theta = \frac{(\mathbf{a}_1 \times \mathbf{a}_3)\cdot(\mathbf{a}_2 \times \mathbf{a}_3)}{|\mathbf{a}_1 \times \mathbf{a}_3| \cdot |\mathbf{a}_2 \times \mathbf{a}_3|}$$

Courtesy: Silvio Savarese

[FP] Sec. 1.3.1

Parameters Extraction

$$\frac{\mathcal{M}}{\rho} = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix} = \mathrm{K}[\mathrm{R} \quad \mathrm{T}]$$

A b

$$\mathrm{A} = \begin{bmatrix} \mathbf{a}_1^{\mathrm{T}} \\ \mathbf{a}_2^{\mathrm{T}} \\ \mathbf{a}_3^{\mathrm{T}} \end{bmatrix} \qquad \mathrm{b} = \begin{bmatrix} \mathrm{b}_1 \\ \mathrm{b}_2 \\ \mathrm{b}_3 \end{bmatrix}$$

Estimated values

Intrinsic

$$\alpha = \rho^2 |\mathbf{a}_1 \times \mathbf{a}_3| \sin\theta$$

$$\beta = \rho^2 |\mathbf{a}_2 \times \mathbf{a}_3| \sin\theta$$

Courtesy: Silvio Savarese

[FP] Sec. 1.3.1

Parameters Extraction

$$\frac{\mathcal{M}}{\rho} = \begin{pmatrix} \alpha \boldsymbol{r}_1^T - \alpha \cot\theta \boldsymbol{r}_2^T + u_0 \boldsymbol{r}_3^T & \alpha t_x - \alpha \cot\theta t_y + u_0 t_z \\ \frac{\beta}{\sin\theta} \boldsymbol{r}_2^T + v_0 \boldsymbol{r}_3^T & \frac{\beta}{\sin\theta} t_y + v_0 t_z \\ \boldsymbol{r}_3^T & t_z \end{pmatrix} = \mathrm{K}[\mathrm{R} \quad \mathrm{T}]$$

A b

$$\mathrm{A} = \begin{bmatrix} \mathbf{a}_1^{\mathrm{T}} \\ \mathbf{a}_2^{\mathrm{T}} \\ \mathbf{a}_3^{\mathrm{T}} \end{bmatrix} \qquad \mathrm{b} = \begin{bmatrix} \mathrm{b}_1 \\ \mathrm{b}_2 \\ \mathrm{b}_3 \end{bmatrix}$$

Estimated values

Extrinsic

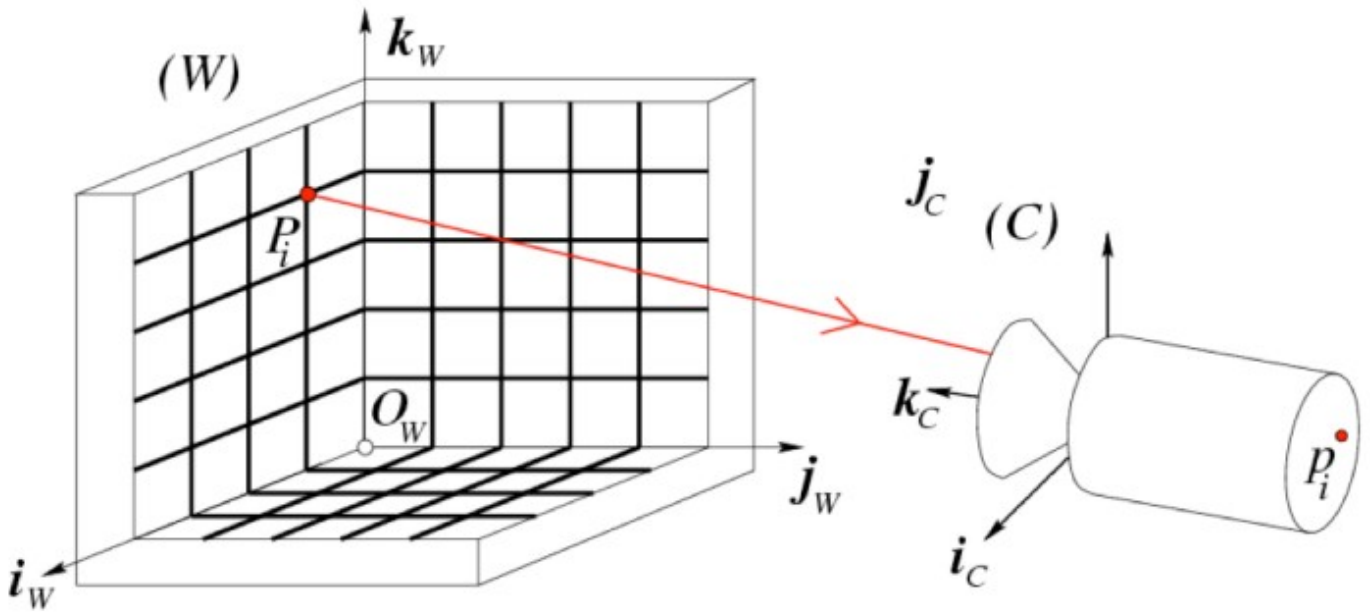
$$\mathbf{r}_1 = \frac{(\mathbf{a}_2 \times \mathbf{a}_3)}{|\mathbf{a}_2 \times \mathbf{a}_3|} \qquad \mathbf{r}_3 = \frac{\pm \mathbf{a}_3}{|\mathbf{a}_3|}$$

$$\mathbf{r}_2 = \mathbf{r}_3 \times \mathbf{r}_1 \qquad \mathrm{T} = \rho \, \mathrm{K}^{-1} \mathbf{b}$$

Courtesy: Silvio Savarese

[FP] Sec. 1.3.1

Degenerate Case



- Not all P_ws lead to a result!
  - P_w points must not lie on the same plane
  - P_w points must not lie on the intersection curve of 2 quadric surfaces

Courtesy: Silvio Savarese

UNIVERSITÀ DI PARMA

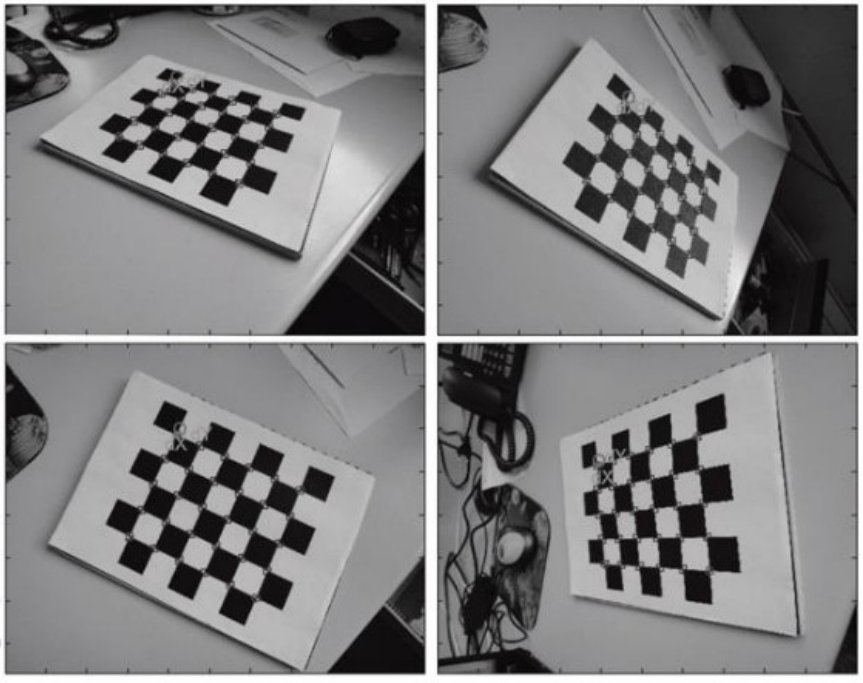
Calibration
Zhang approach (2000)

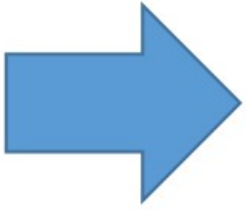
Zhang approach (2000)

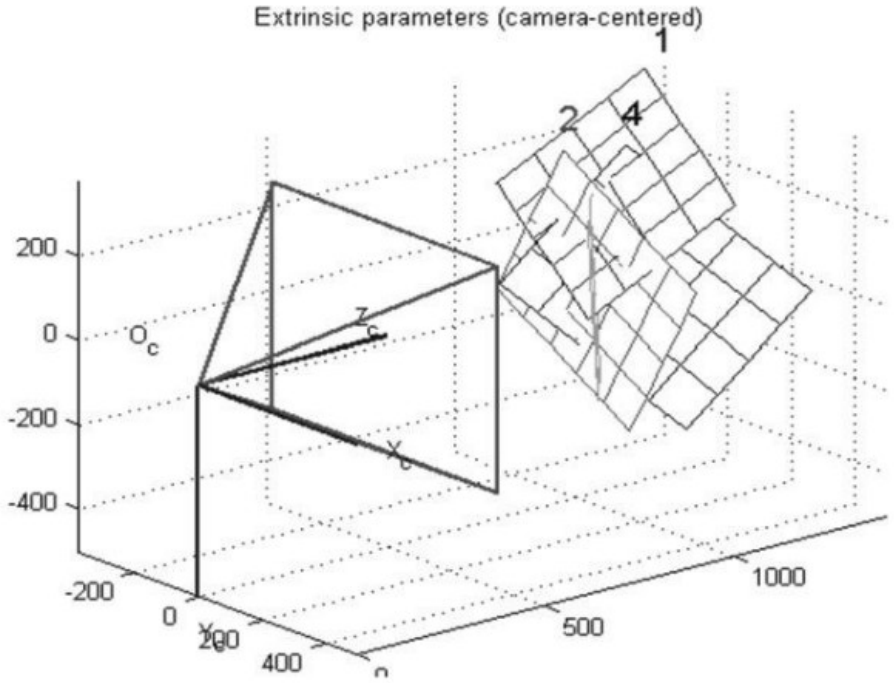
UNIVERSITÀ DI PARMA

- Not very practical to use 3D calibration rigs…
- Usually calibration grids are “flat” (Matlab, OpenCV)
- This does not fit well with Tsai’s requirements
  - No coplanar points
- Zhang’s approach, conversely, exploits this!

Zhang approach (2000)

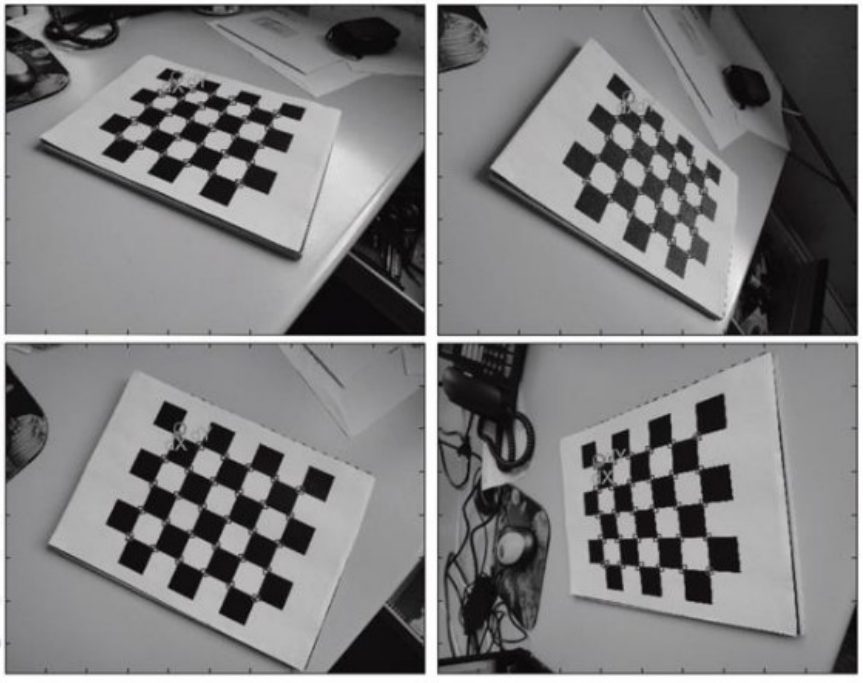


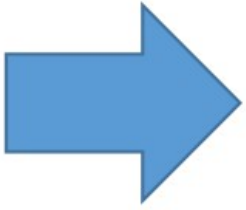


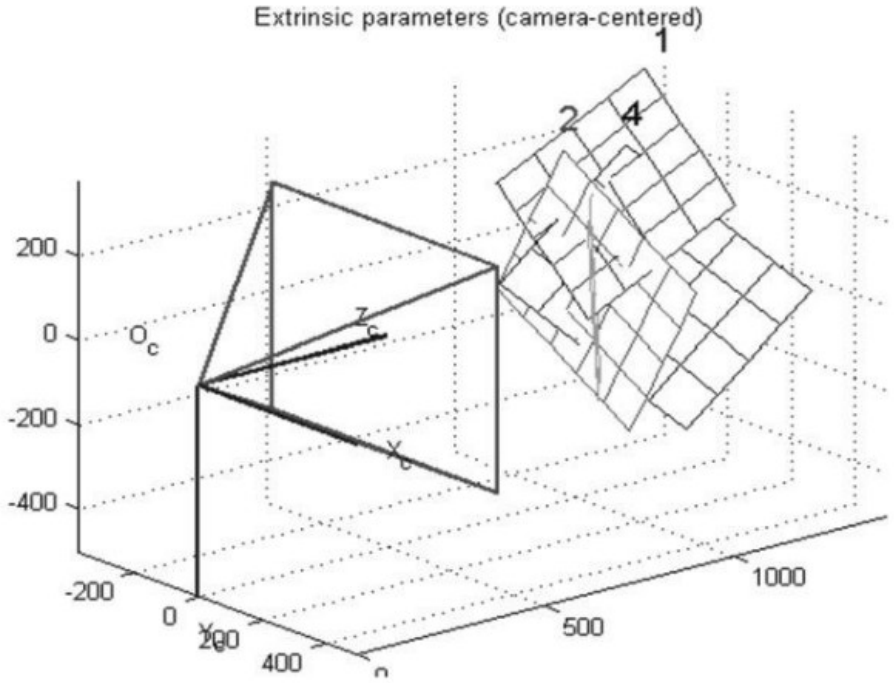


Zhang, *A flexible new technique for camera calibration*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 2000.

Zhang approach (2000)

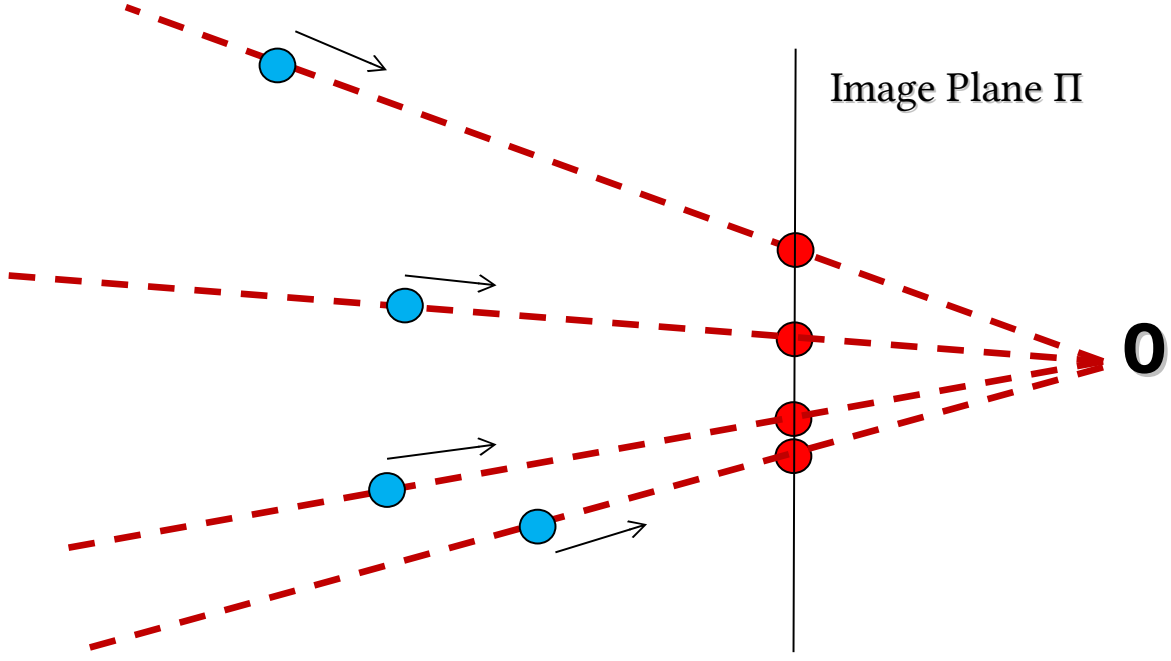






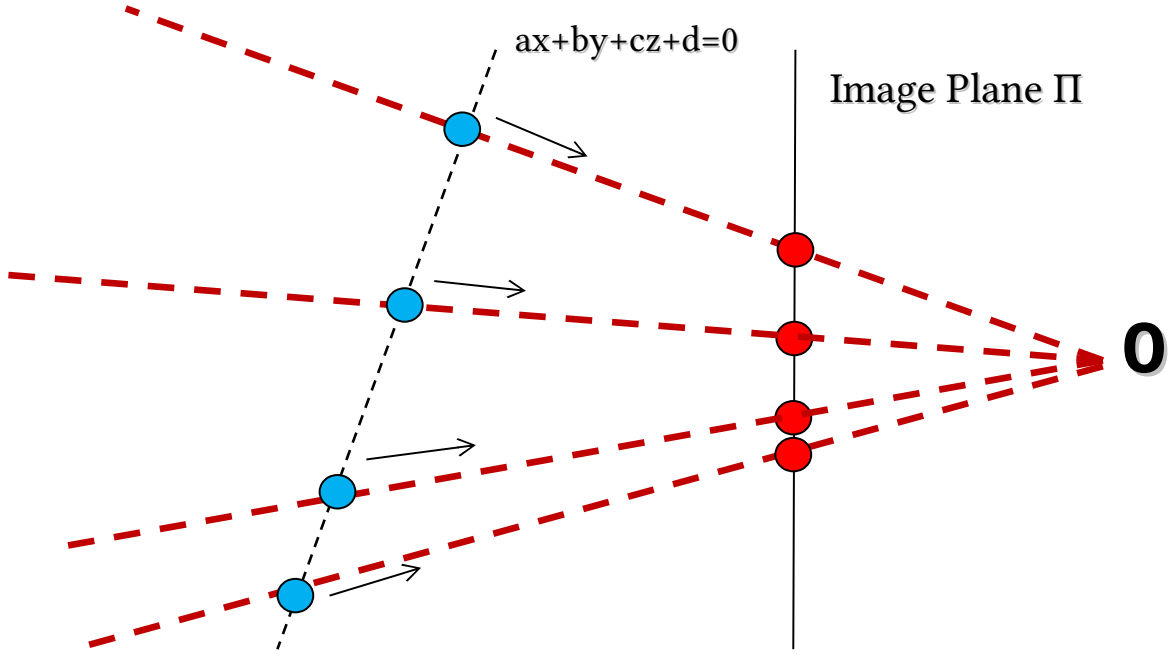
Before detailing Zhang's approach, just understand what does it mean to frame a "plane"

Ill posed problem



- Projective space ⟶ we lost one dimension
- 3D [x,y,z] are projected in [x/z,y/z,1]

Add a constraint



- Just image that the (red) image points are projection of points that lie on the same plane
- In such a case for each image point there is only one world point (cyan)

Add a constraint

UNIVERSITÀ DI PARMA

- Given a plane aX + bY + cZ + d = 0

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} \overset{3\times4}{=} M \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad \rightarrow \quad \begin{bmatrix} u \\ v \\ w \\ 0 \end{bmatrix} = \overset{4\times4}{\begin{bmatrix} M \\ a\,b\,c\,d \end{bmatrix}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} = \begin{bmatrix} M \\ a\,b\,c\,d \end{bmatrix}^{-1} \begin{bmatrix} u \\ v \\ 1 \\ 0 \end{bmatrix} \quad \overset{euclidean}{\rightarrow} \quad \begin{bmatrix} X/W \\ Y/W \\ Z/W \end{bmatrix}$$

- We can obtain euclidean world coordinates of pixel (u,v)

Specific case, plane Z=0

UNIVERSITÀ DI PARMA

- Consider a very specific plane → Z=0

$$\begin{bmatrix} u \\ v \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \\ m_{41} & m_{42} & m_{43} & m_{44} \end{bmatrix} \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix}$$

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = H \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

- H is 3×3
- H is an **Homography**
- This actually works for every possible plane!

Better understanding H

UNIVERSITÀ DI PARMA

- We started from a general M but
  - M = K [R T]

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \overbrace{\begin{bmatrix} \alpha & -\alpha\cot\theta & c_x \\ 0 & \dfrac{\beta}{\sin\theta} & c_y \\ 0 & 0 & 1 \end{bmatrix}}^{K_{3\times 3}} \overbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}}^{[RT]_{3\times 4}} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Better understanding H

UNIVERSITÀ DI PARMA

- When the Z = 0 we can simplify as:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ r_{31} & r_{32} & t_z \end{bmatrix}_{3\times 3} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

Better understanding H

UNIVERSITÀ DI PARMA

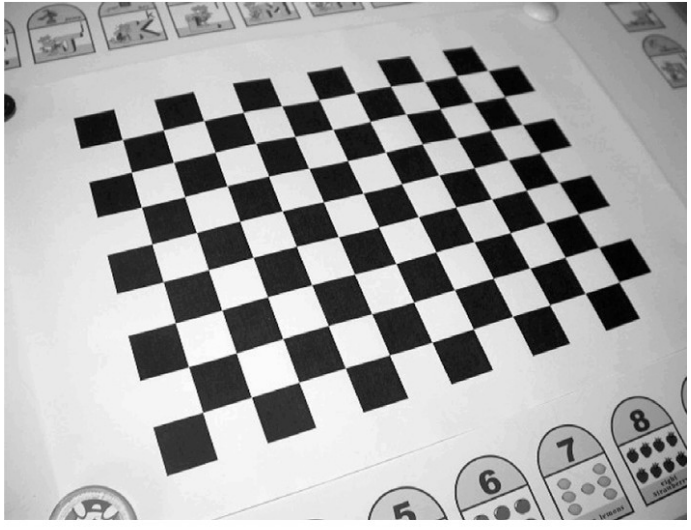
- When the Z = 0 we can simplify as:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} r_{11} & r_{12} & t_x \\ r_{21} & r_{22} & t_y \\ r_{31} & r_{32} & t_z \end{bmatrix}_{3\times 3} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = \overbrace{K \begin{bmatrix} r_1 r_2 t \end{bmatrix}}^{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}$$

Zhang's approach

UNIVERSITÀ DI PARMA

- Zhang idea is to use a "planar" calibration checkboard
  - Set the world coordinate system onto the checkboard ⟶ z = 0 plane…
- In such a case we can apply an homography!



Calibration

UNIVERSITÀ DI PARMA

- For each point on the planar surface of my grid I then obtain
- In the same image…

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = H \begin{bmatrix} X_i \\ Y_i \\ 1 \end{bmatrix} \quad \text{with} \quad i = 1, 2, 3, \ldots, n$$

Calibration

UNIVERSITÀ DI PARMA

- Again a system of n equations
  - Unknowns are H elements

$$QH = 0$$

Calibration

UNIVERSITÀ DI PARMA

- For solving this issue we can use the same approach used for “Tsai”
- We have a 3×3 homograpy instead of a 3×4 projection matrix
- We need to solve $QH = 0$ where Q is $2n \times 9$ and has rank 8
- We need at least 4 points
  - When $n > 4 \longrightarrow$ SVD

Calibration

UNIVERSITÀ DI PARMA

- For computing H we can, again, use a SVD
- It is, again, an homogeneous system

$$QH = 0$$

- Is H really useful?
  - No, given they mix intrinsic parameters and very specific extrinsic ones
  - I put my coords system on the grid!

Computing H

UNIVERSITÀ DI PARMA

- Each view has different H but **K is the same for all views**

$$H^i = K\left[r_1^i \, r_2^i \, t\right]$$

- Once computed H^i we can focus on $H^i \leftrightarrow K$ relation

Computing K given H

UNIVERSITÀ DI PARMA

- K can be anyway computed from H
  - Constraints: r_1, r_2 form an orthonormal basis
    - This allows to compute$K^{-T}K^{-1}$
  - Cholesky decomposition can be applied to the symmetric and positive $B=K^{-T}K^{-1}$ for computing K^T
  - Constraints: we need multiple views
    - We will discuss this later (without details)

Computing extrinsic parameters

UNIVERSITÀ DI PARMA

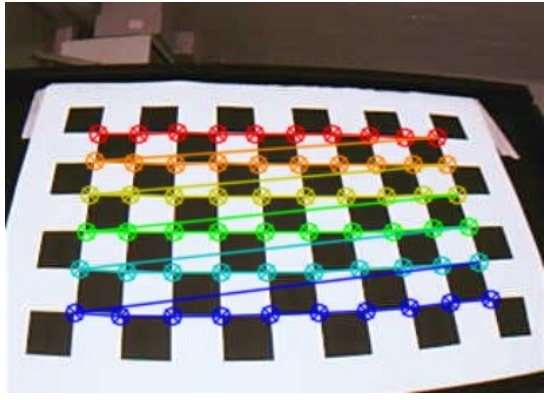
- We now have multiple H and one K
- We can compute extrinsic parameters of each view from them…
  - A scale factor is, again, to be considered…
  - λ

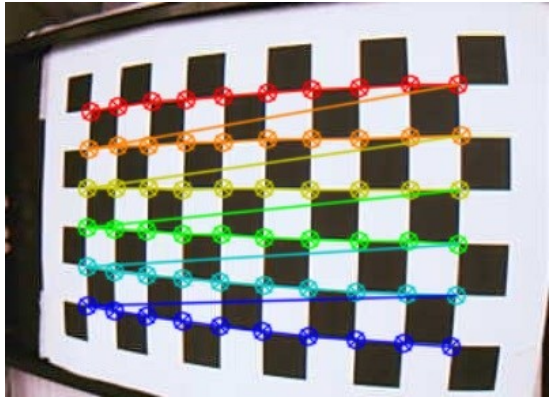
$$
\begin{aligned}
r_1 &= \lambda K^{-1} h_1 \\
r_2 &= \lambda K^{-1} h_2 \\
r_3 &= r_1 \times r_2 \\
t &= \lambda K^{-1} h_3
\end{aligned}
$$

How many grids?

UNIVERSITÀ DI PARMA

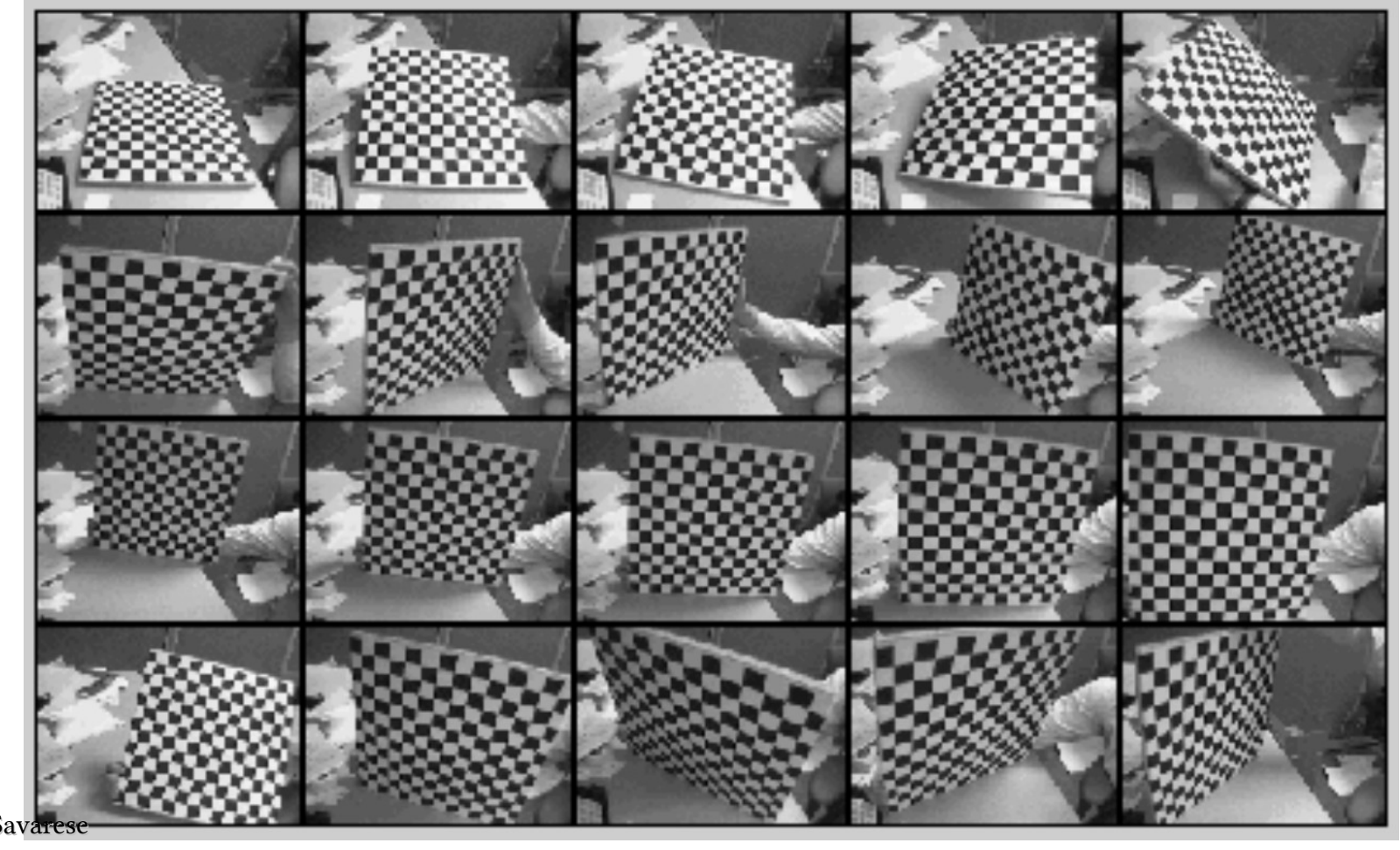
- With a “single” H we can not anyway fully compute K
- 3 Hs are needed, at least, to compute K
  - The larger the number of views, the better the result
  - Usually 20-50 grid views





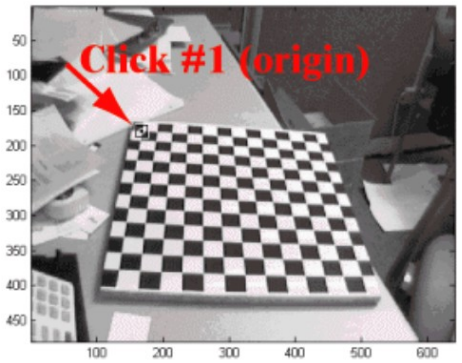
Example of calibration data

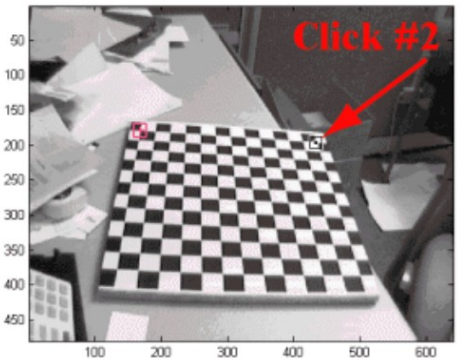
UNIVERSITÀ DI PARMA

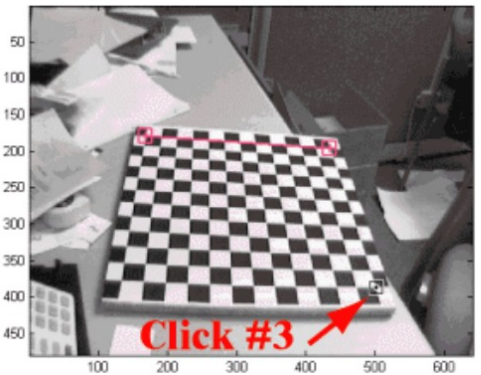


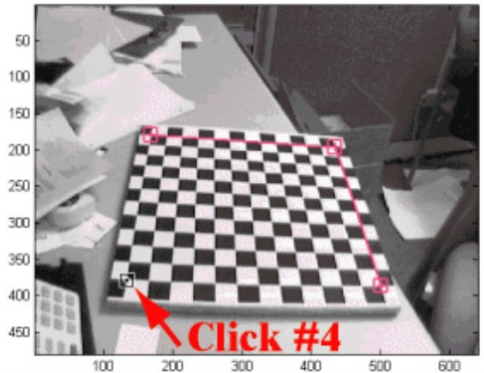
Courtesy: Silvio Savarese

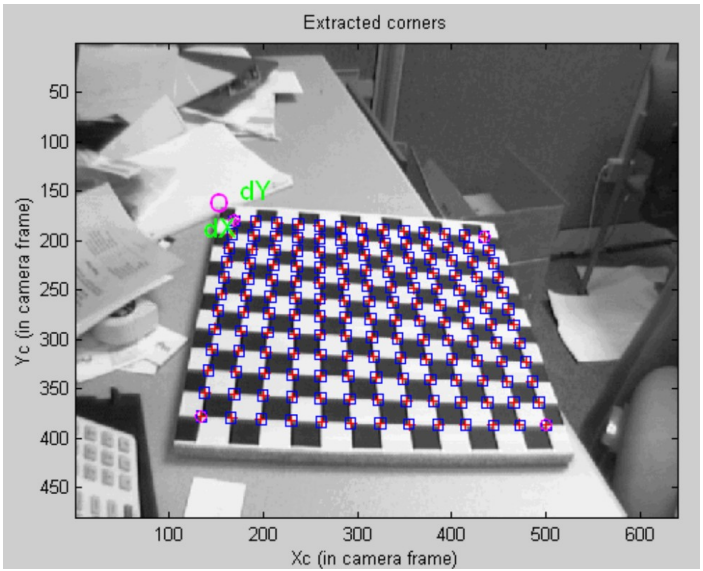
Example of calibration data









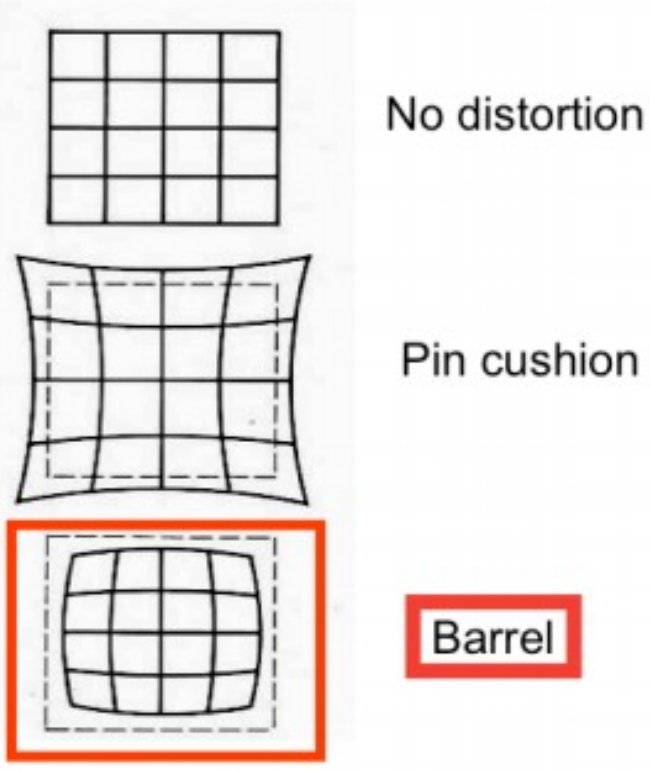


Courtesy: Silvio Savarese

UNIVERSITÀ DI PARMA

Lens Distortion

Lens Distortion

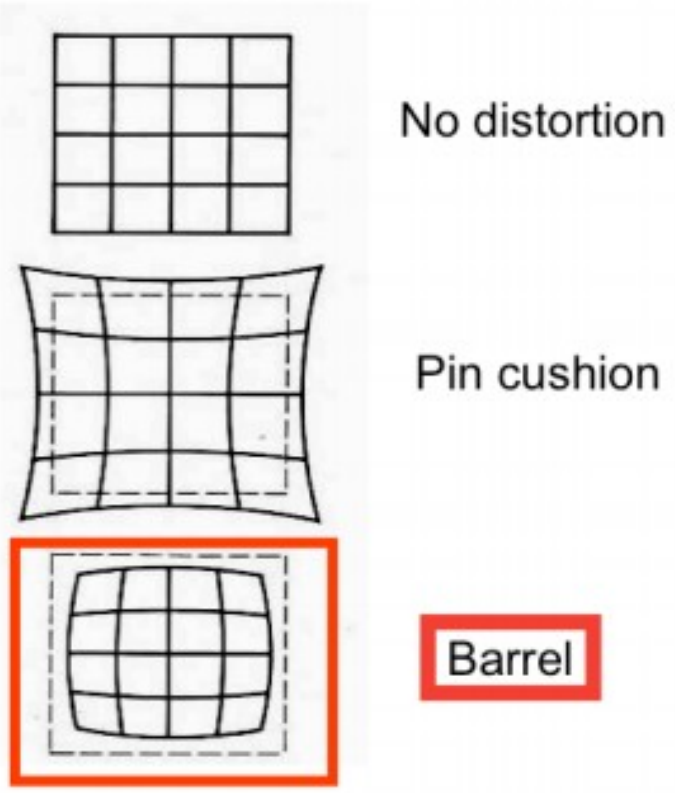


Courtesy: Silvio Savarese

Radial Lens Distortion

UNIVERSITÀ DI PARMA

- We assumed a “thin” lens
  - This basically means that lines are lines
  - Unfortunately this is not true!
- Radial Distortion
  - Image magnification/contraction depends on distance from optical axis
  - Deviations are more evident on lateral areas
  - Cheap optics

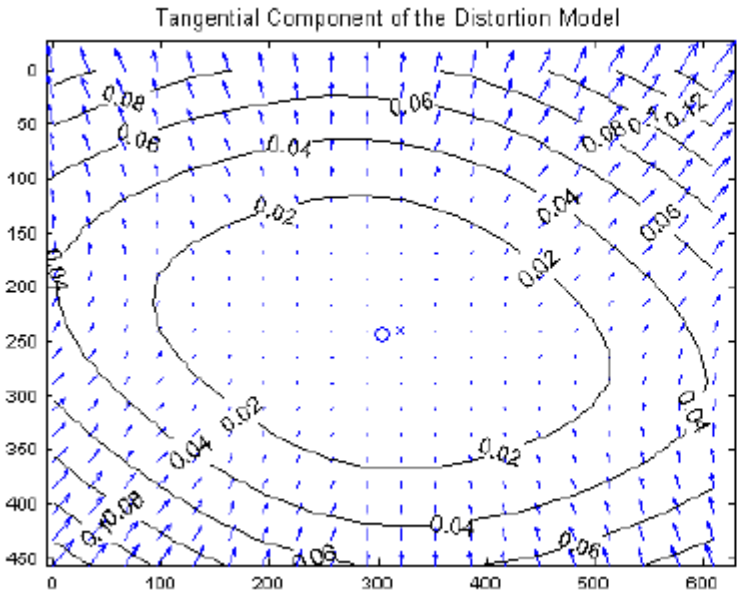


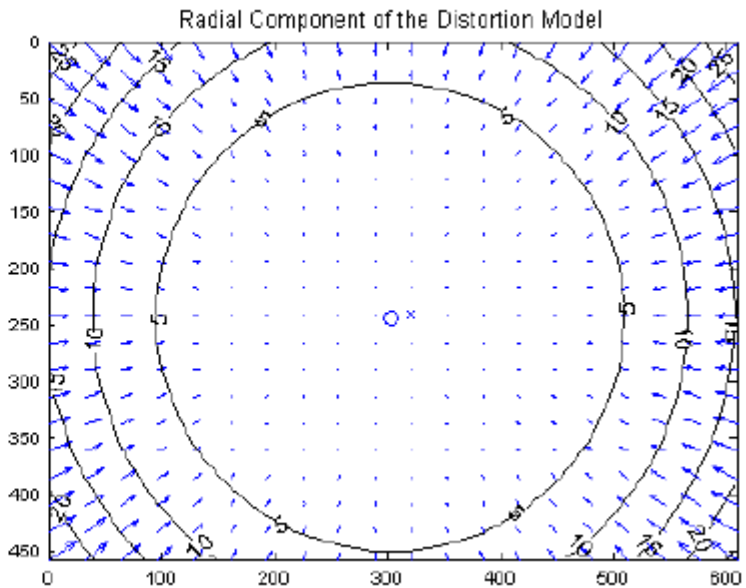
Courtesy: Silvio Savarese

Tangental Lens Distortion

UNIVERSITÀ DI PARMA

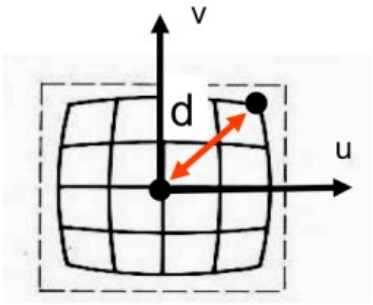
- Typical of misaligned lenses
  - Not parallel to image sensor





Courtesy: Technische Universität München

Radial Lens Distortion



$$S_\lambda \begin{bmatrix} \frac{1}{\lambda} & 0 & 0 \\ 0 & \frac{1}{\lambda} & 0 \\ 0 & 0 & 1 \end{bmatrix} M P_i \rightarrow \begin{bmatrix} u_i \\ v_i \end{bmatrix} = p_i$$

Distortion coefficient

$$\lambda = 1 \pm \sum_{p=1}^{3} \kappa_p d^{2p} \qquad d^2 = a\,u^2 + b\,v^2 + c\,u\,v$$

[Eq. 5] Polynomial function — To model radial behavior [Eq. 6]

- λ is the distortion factor
  - Actually it is a $\lambda_{(u,v)}$ since it depends on the distance from optical axis

Courtesy: Silvio Savarese

Radial Lens Distortion

$$\underbrace{\begin{bmatrix} \frac{1}{\lambda} & 0 & 0 \\ 0 & \frac{1}{\lambda} & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{Q} M P_i \rightarrow \begin{bmatrix} u_i \\ v_i \end{bmatrix} = p_i \qquad Q = \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \mathbf{q}_3 \end{bmatrix}$$

- λ depends on u,v and this leads to a non linearity in the $P \rightarrow p$ mapping

Courtesy: Silvio Savarese

Radial Lens Distortion

$$\underbrace{\begin{bmatrix} \frac{1}{\lambda} & 0 & 0 \\ 0 & \frac{1}{\lambda} & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{Q} M \quad P_i \rightarrow \begin{bmatrix} u_i \\ v_i \end{bmatrix} = p_i \qquad Q = \begin{bmatrix} \mathbf{q}_1 \\ \mathbf{q}_2 \\ \mathbf{q}_3 \end{bmatrix}$$

- Tsai 87, initially estimate m_1 and m_2 by SVD and then $m_3=f(m_1, m_2, \lambda)$

Courtesy: Silvio Savarese

UNIVERSITÀ DI PARMA

OpenCV Calibration

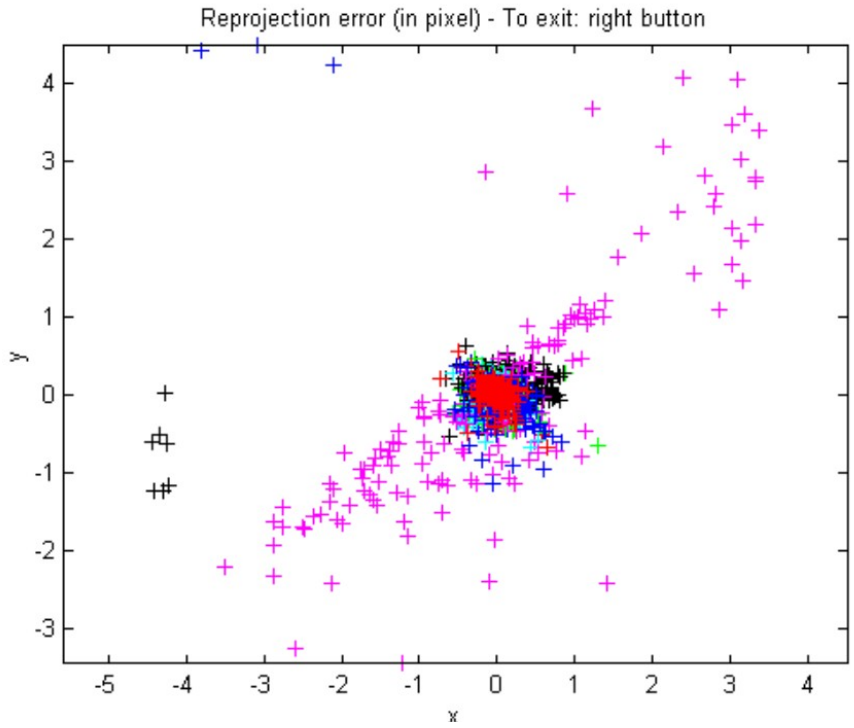
Multi plane calibration

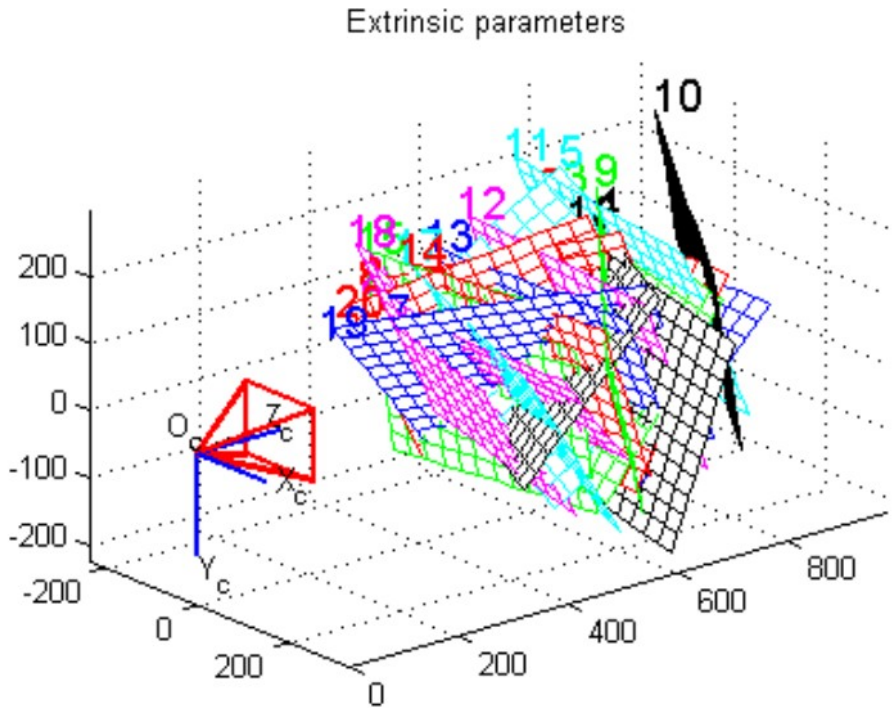
UNIVERSITÀ DI PARMA

- Requires a plane only!
- No known positions/orientations
- Strobl et al. approach (2011)

- Free code:
  - https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

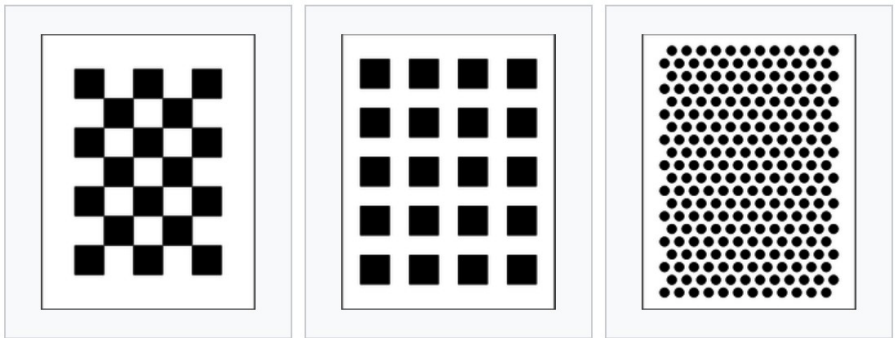
Example of calibration data

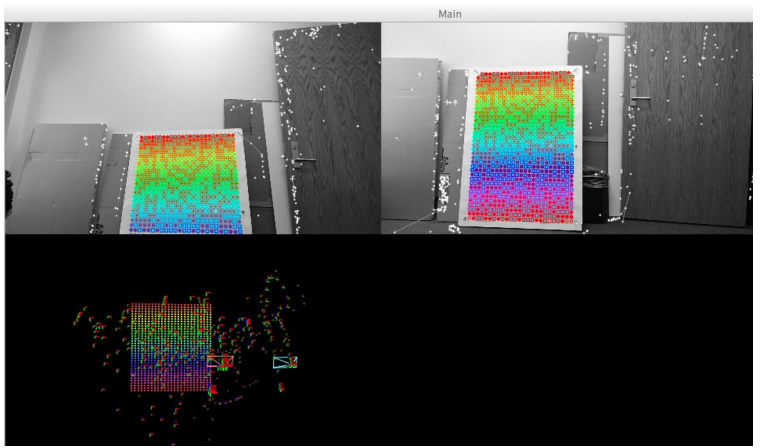




Courtesy: Silvio Savarese

Calibration Patterns





https://github.com/arpg/Documentation/tree/master/Calibration

https://boofcv.org/index.php?title=Tutorial_Camera_Calibration

UNIVERSITÀ DI PARMA

Perspective-n-Point

Extrinsic parameters

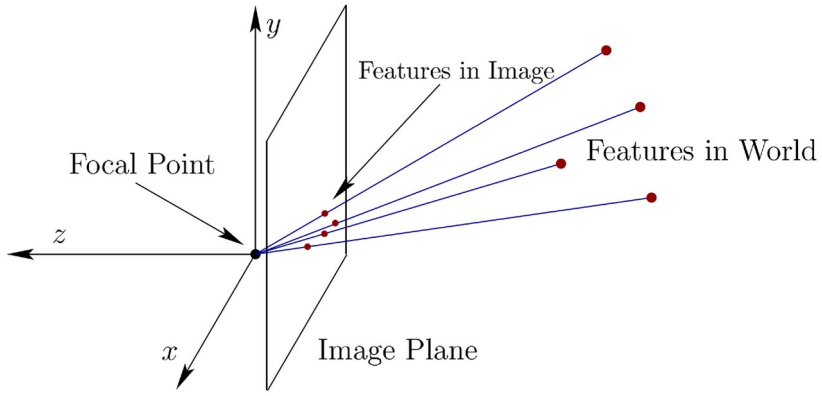
UNIVERSITÀ DI PARMA

- Tsaj, Zhang, and other methods allow to easily compute K
  - Intrinsic parameters “only”
- General problem:
  - I already computed K
  - I want to compute or update [RT]
  - 6 DOF (3 translation, 3 rotation)
- Namely, camera pose estimation “only”

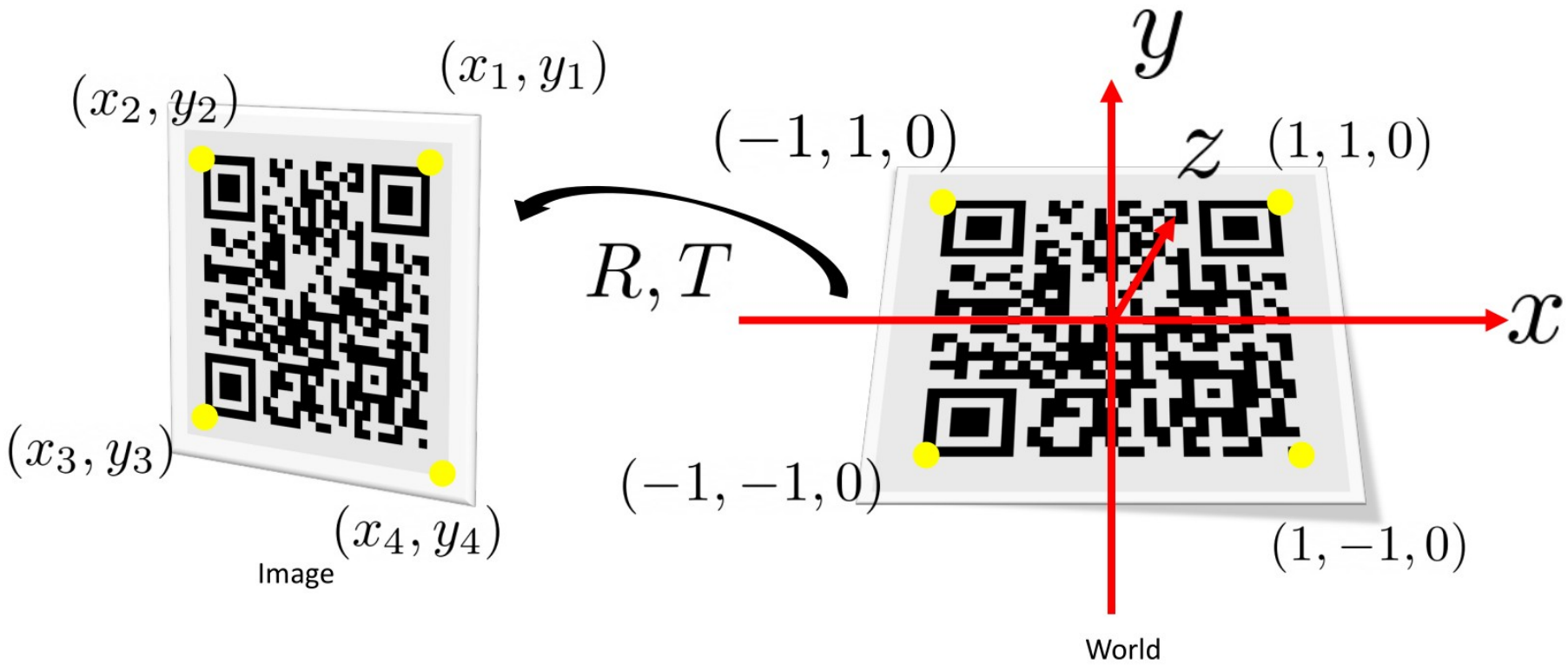
Perspective-n-Point

UNIVERSITÀ DI PARMA

- We know:
  - A set of *n* world coordinates P_w
  - Their projections p_c on an image
  - Camera Intrinsics K
- Unknowns
  - 3D camera rotation wrt world coordinates R
  - 3D camera translation wrt wolrd reference origin
- Different approaches



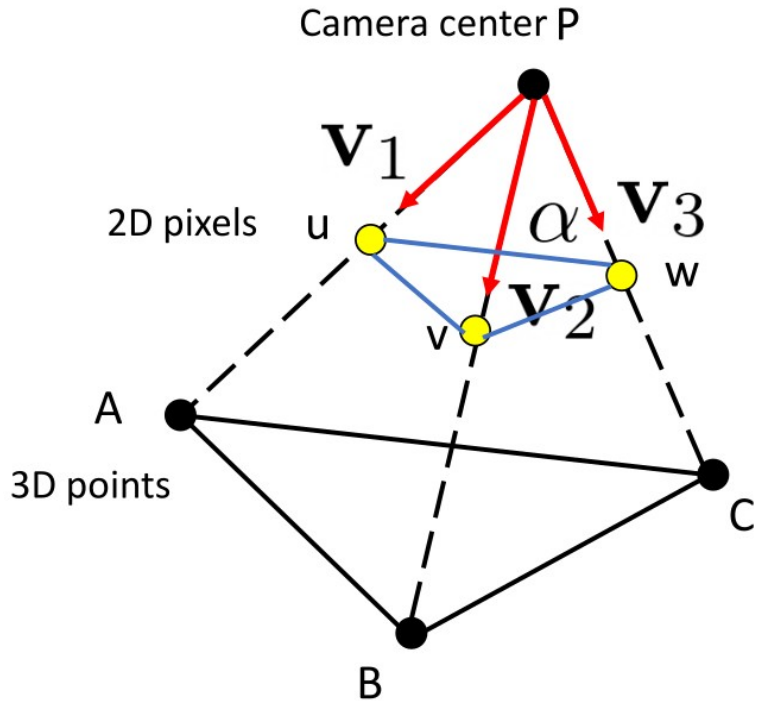
PnP Problem with QR code



Courtesy: Yu Xiang

- I can use only 3 known points (A, B, C)
- And their projections u, v, w

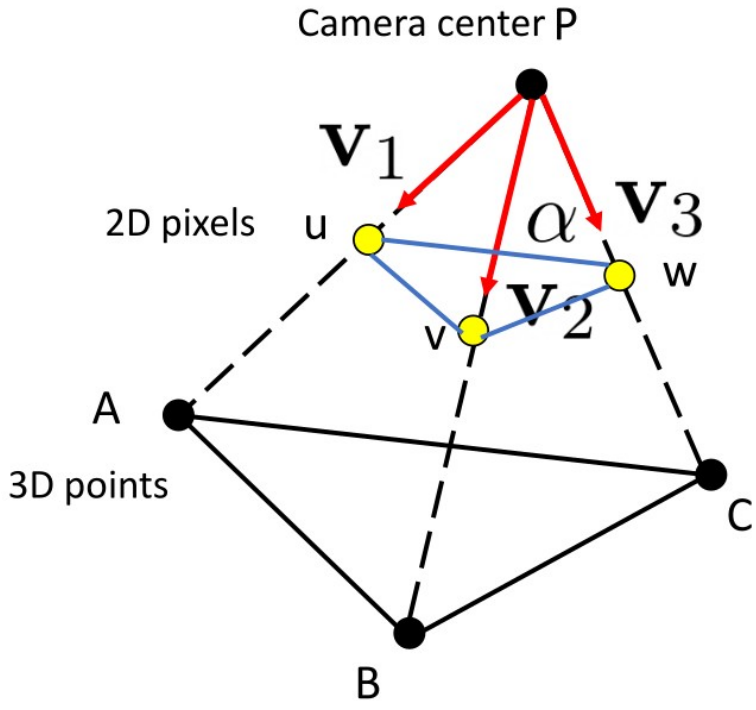
$$
\begin{aligned}
u &= K[I\,0]A \\
v &= K[I\,0]B \\
w &= K[I\,0]C
\end{aligned}
$$



Courtesy: Yu Xiang

- K^{-1} can be computed for sure
- If I multiply K^{-1} for u, v, w what is the result?
  - Lines of sight

$$\vec{v}_1 = K^{-1}u$$
$$\vec{v}_2 = K^{-1}v$$
$$\vec{v}_3 = K^{-1}w$$

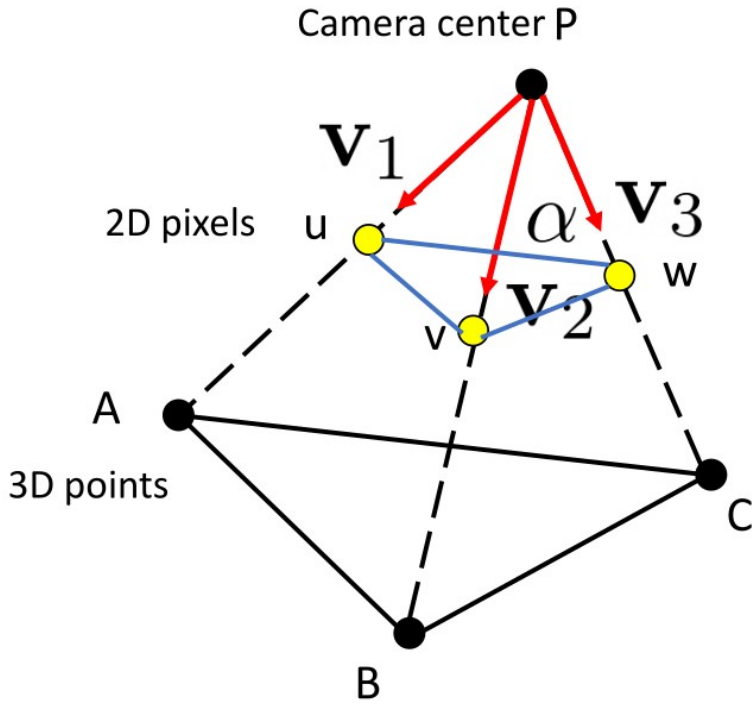


Courtesy: Yu Xiang

- Using the 3 vectors we can also compute the angles between them
- As example:

$$\cos(\alpha) = \frac{\vec{v}_2 \cdot \vec{v}_3}{\|v_2\| \ \|v_3\|}$$

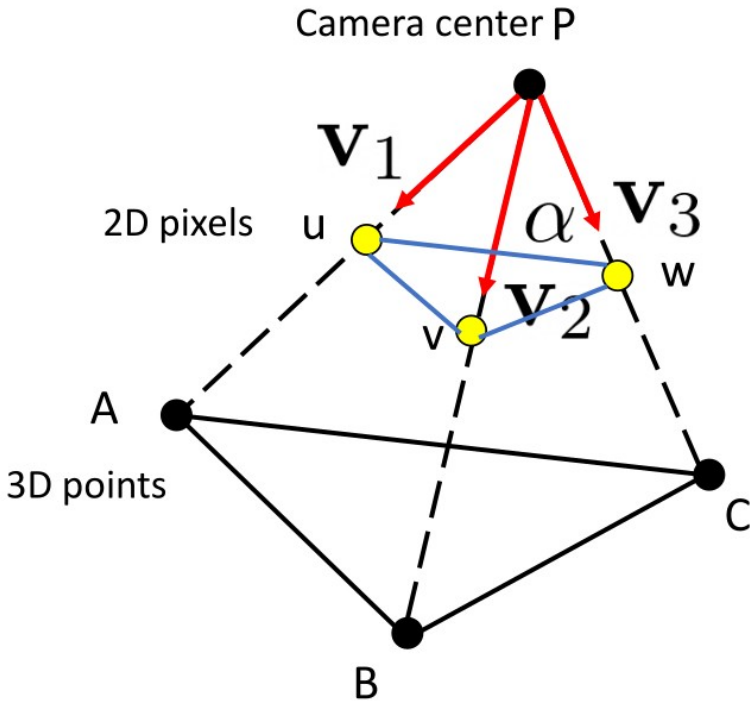
- The same for other angles (γ and β)



Courtesy: Yu Xiang

- We can compute the distance of points A, B, and C from P exploiting:

$$\overline{AB}^2 = \overline{PA}^2 + \overline{PB}^2 - 2\cos(\gamma)$$
$$\overline{AC}^2 = \overline{PA}^2 + \overline{PC}^2 - 2\cos(\beta)$$
$$\overline{BC}^2 = \overline{PB}^2 + \overline{PC}^2 - 2\cos(\alpha)$$

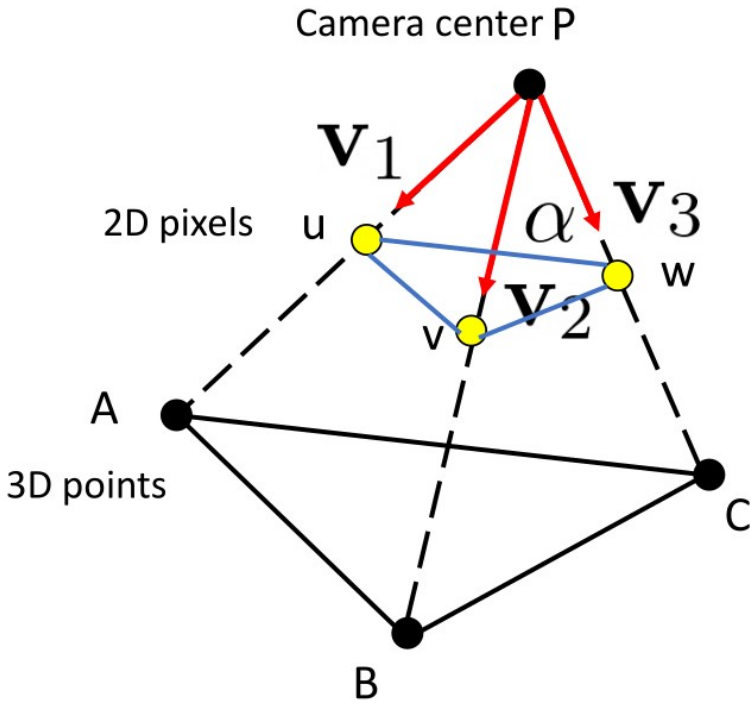


- PA, PB, and PC are the unknowns!

Courtesy: Yu Xiang

P3P

- We can solve previous equation system but we obtain 4 possible solutions…
- A 4th point can be used which one aligns most accurately
  - We should rather say P3+1P
  - No exact solution for sure (noise)
- Given the relative position between P and the 3 world points we can recover camera pose



Courtesy: Yu Xiang

P3P with RANSAC

UNIVERSITÀ DI PARMA

- RANSAC is a method to estimate parameters from noisy observed data
- Yes, our measurement are for sure noisy!
- Instead of using 3+1 points, use N points where N >> 3
  - Randomly select 3 points
  - Use P3P to compute camera pose ⟶ current hypothesis
  - Count how many points support current hypothesis
  - Go back to point selection until we found a suitable solution or sufficient trials

EPnP

- An accurate solution to the PnP problem
  - E ⟶ efficient
  - Non iterative
  - O(n) wrt $O(n^x)$ with $5 \leq x \leq 8$
  - Slightly less precise than iterative methods
  - ICCV 2007

- cv::solvePnP(), different methods:
  - cv::SOLVEPNP_P3P or cv::SOLVEPNP_AP3P: 4 input points
  - cv::SOLVEPNP_IPPE: >= 4 coplanar points
  - cv::SOLVEPNP_IPPE_SQUARE: 4 coplanar points on thje corners of a square (QR code)
- cv::solvePnPRansac(): more than 4 points
- Other refinement methods

UNIVERSITÀ DI PARMA

Camera Models (2)

Question time!